

NOSQL PROJEKAT

KOTVA

Informacioni sistem za osiguravajuće društvo

Node.js · Express · ArangoDB · Chart.js

Kompletna dokumentacija projekta · Avgust 2026.

Sadržaj

- | | |
|-------------------------------|--|
| 1. OPIS PROJEKTA | 15. VALIDACIJA PODATAKA |
| 2. GLAVNE FUNKCIONALNOSTI | 16. ANALITIKA NA FRONTENDU |
| 3. ARHITEKTURA SISTEMA | 17. PRIMERI AQL UPITA |
| 4. POKRETANJE PROJEKTA | 18. VREMENSKA I MEMORIJSKA
KOMPLEKSNOST |
| 5. DOCKER KONFIGURACIJA | 19. MOGUĆNOSTI OPTIMIZACIJE |
| 6. NOSQL I ARANGODB | 20. SKALIRANJE SISTEMA |
| 7. KOLEKCIJE | 21. CAP, ACID I BASE |
| 8. PRIMER DOKUMENTA KORISNIKA | 22. NORMALIZACIJA I DENORMALIZACIJA |
| 9. PRIMER DOKUMENTA POLISE | 23. BEZBEDNOST |
| 10. GRAPH MODEL | 24. BACKUP I TRAJNOST |
| 11. GRAPH TRAVERSAL | 25. PLAN PREZENTACIJE |
| 12. ANALYZER I PRETRAGA | 26. KRATAK AQL UPIT ZA PREZENTACIJU |
| 13. INDEKSI | 27. ZAKLJUČAK |
| 14. API RUTE | |

Verzija dokumentacije: 31. avgust 2026.

Naziv projekta: Kotva

Kotva · NoSQL dokumentacija · ArangoDB

Tip projekta: NoSQL web aplikacija

Baza podataka: ArangoDB 3.12

Backend: Node.js i Express

Frontend: HTML, CSS, JavaScript i Chart.js

1. OPIS PROJEKTA

Kotva je informacioni sistem za evidentiranje osiguranika, polisa i osiguravajućih kuća. Sistem omogućava unos novih korisnika i osiguravajućih kuća, čuvanje podataka u ArangoDB bazi, pretragu korisnika i prikaz poslovne analitike pomoću grafikona.

Podržane vrste osiguranja su:

- Putno osiguranje
- Životno osiguranje
- Auto-osiguranje
- Osiguranje privatne svojine

Početne osiguravajuće kuće su:

- Uniqa
- Generali
- Milenijum
- Sava

Sistem je proširiv, pa se nove kuće i novi korisnici mogu unositi preko frontenda. Grafikoni se automatski osvežavaju nakon unosa.

2. GLAVNE FUNKCIONALNOSTI

- Unos imena i prezimena korisnika
- Unos godina korisnika
- Izbor vrste osiguranja
- Izbor osiguravajuće kuće
- Unos datuma prodaje polise
- Dodavanje novih osiguravajućih kuća
- Prikaz poslednjih korisnika
- Pretraga korisnika po imenu ili prezimenu
- Pretraga bez obzira na velika slova i dijakritike

- Automatsko kreiranje polise i graph veza za novog korisnika
- Analitika i interaktivni grafikoni Kotva · NoSQL dokumentacija · ArangoDB
- ArangoDB web interfejs
- Trajno čuvanje podataka u Docker volumenu
- Rezervni režim rada u memoriji kada ArangoDB nije dostupan

3. ARHITEKTURA SISTEMA

Sistem je podeljen na tri sloja:

1. Frontend

- public/index.html
- public/styles.css
- public/app.js

2. Backend API

- server.js
- Node.js
- Express
- arangojs drajver

3. Baza podataka

- ArangoDB
- document kolekcije
- edge kolekcije
- named graph
- analyzer
- persistent i inverted indeksi

Tok zahteva:

```
Korisnik -> Frontend -> Express API -> ArangoDB -> Express API -> Frontend
```

Frontend šalje HTTP zahtev backendu. Backend validira podatke, izvršava operaciju nad bazom i vraća JSON odgovor. Frontend zatim ažurira tabelu, statistiku i grafikone.

4. POKRETANJE PROJEKTA

Potrebni alati:

- Docker Desktop
- Docker Compose
- Web pregledač

U PowerShell terminalu otvoriti direktorijum projekta:

```
cd C:\Users\uros1\Desktop\kotva
```

Kotva · NoSQL dokumentacija · ArangoDB

Pokrenuti kontejnere:

```
docker compose up --build -d
```

Aplikacija je dostupna na:

```
http://localhost:3001
```

ArangoDB web interfejs je dostupan na:

```
http://localhost:8529
```

Podaci za prijavu:

Korisnik: root

Lozinka: kotva123

Baza: kotva

Provera kontejnera:

```
docker compose ps
```

Pregled logova:

```
docker compose logs -f
```

Zaustavljanje sistema bez brisanja podataka:

```
docker compose down
```

VAŽNO: Komanda "docker compose down -v" briše Docker volume i podatke baze.

5. DOCKER KONFIGURACIJA

Docker Compose pokreće dva servisa:

1. kotva-arangodb

- ArangoDB 3.12
- host port 8529
- trajni volume arango_data
- healthcheck preko arangosh komande

2. kotva-app

- Node.js aplikacija
- interni port 3000
- host port 3001
- čeka da ArangoDB postane healthy

Razlika između portova:

- Node aplikacija unutar kontejnera sluša na portu 3000.
- Na Windows host računaru aplikacija je izložena na portu 3001.

Zemljana NoSQL Dokumentacija · ArangoDB

6. NOSQL I ARANGODB

NoSQL označava baze koje nisu ograničene isključivo na relacioni model tabela.

ArangoDB je multi-model NoSQL baza koja podržava:

- document model
- key-value pristup
- graph model
- full-text pretragu
- AQL upite

Dokumenti su JSON-slične strukture. ArangoDB je schema-free, što znači da svi dokumenti ne moraju imati potpuno identična polja. Ipak, aplikacija koristi backend validaciju kako bi održala kvalitet podataka.

7. KOLEKCIJE

Document kolekcije:

1. clients

Čuva korisnike i osnovne informacije o njihovom osiguranju.

2. insurers

Čuva osiguravajuće kuće.

3. policies

Čuva zasebne dokumente polisa.

Edge kolekcije:

4. owns

Povezuje korisnika sa polisom.

5. issued_by

Povezuje polisu sa osiguravajućom kućom.

8. PRIMER DOKUMENTA KORISNIKA

```
{
  "_key": "korisnik-1",
  "_id": "clients/korisnik-1",
```

```
"_rev": "revizija-dokumenta",
```

Kotva · NoSQL dokumentacija · ArangoDB

```
"id": "korisnik-1",
```

```
"name": "Ana",
```

```
"surname": "Marković",
```

```
"age": 29,
```

```
"insuranceType": "Putno",
```

```
"insurer": "Unika",
```

```
"saleDate": "2026-01-15"
```

```
}
```

Sistemska polja:

- `_key`: jedinstveni ključ dokumenta unutar kolekcije
- `_id`: naziv kolekcije i ključ dokumenta
- `_rev`: identifikator trenutne revizije dokumenta

Polje `_rev` se menja prilikom izmene dokumenta i može se koristiti za optimistic concurrency control, odnosno sprečavanje prepisivanja tuđih izmena.

9. PRIMER DOKUMENTA POLISE

```
{
```

```
"_key": "polisa-korisnik-1",
```

```
"clientId": "korisnik-1",
```

```
"insuranceType": "Putno",
```

```
"saleDate": "2026-01-15",
```

```
"validFrom": "2026-01-15",
```

```
"validUntil": "2027-01-15",
```

```
"premium": 4225,
```

```
"status": "Aktivna"
```

```
}
```

Premija se trenutno izračunava demonstracionom formulom na osnovu vrste osiguranja i godina korisnika.

10. GRAPH MODEL

Kotva · NoSQL dokumentacija · ArangoDB

Naziv grafa:

kotva_insurance_graph

Struktura:

```
clients --owns--> policies --issued_by--> insurers
```

Primer owns veze:

```
{
  "_from": "clients/korisnik-1",
  "_to": "policies/polisa-korisnik-1"
}
```

Primer issued_by veze:

```
{
  "_from": "policies/polisa-korisnik-1",
  "_to": "insurers/kuca-1"
}
```

Graph model omogućava da jedan korisnik u budućnosti ima više polisa i da se efikasno pronalaze sve polise određene kuće ili korisnika.

11. GRAPH TRAVERSAL

Graph traversal je obilazak povezanih čvorova praćenjem edge veza.

Smerovi:

- OUTBOUND prati smer `_from -> _to`
- INBOUND prati smer `_to -> _from`
- ANY ignoriše smer veze

Primer pronalaženja polisa korisnika:

```
FOR policy IN 1..1 OUTBOUND "clients/korisnik-1" owns
  RETURN policy
```

Brojevi 1..1 znače da se obilazi tačno jedna veza u dubinu.

Primer obilaska named grafa do dubine dva:

```
FOR vertex, edge, path
  IN 1..2 OUTBOUND "clients/korisnik-1"
```



```
GRAPH "kotva_insurance_graph"
```

Kotva · NoSQL dokumentacija · ArangoDB

```
RETURN {
```

vertex: vertex,

edge: edge,

path: path

```
}
```

12. ANALYZER I PRETRAGA

Analyzer:

kotva::sr_name_search

Tip:

norm

Podešavanja:

- locale: sr
- case: lower
- accent: false

Analyzer normalizuje tekst, tako da pretraga "Markovic" može pronaći

```
"Marković". Koristi se zajedno sa inverted indeksom i SearchAlias view-om.
```

Inverted indeks:

idx_client_name_search

Indeksirana polja:

- name
- surname

Search view:

client_search

Frontend koristi API rutu:

```
GET /api/search?q=Markovic
```

13. INDEKSI

Primary indeks:

- automatski postoji
- indeksira _key
- jedinstven je

Persistent indeks:

idx_insurance_type_sale_date

Kotva · NoSQL dokumentacija · ArangoDB

Polja redom:

1. insuranceType

2. saleDate

Indeks je pogodan za upite koji koriste jednakost nad insuranceType i opseg ili sortiranje nad saleDate.

Primer:

```
FOR client IN clients
  FILTER client.insuranceType == "Putno"
  FILTER client.saleDate >= "2026-07-01"
  FILTER client.saleDate < "2026-08-01"
  SORT client.saleDate
  RETURN {
```

korisnik: CONCAT(client.name, " ", client.surname),

datumProdaje: client.saleDate,

osiguravajucaKuca: client.insurer

```
}
```

U Explain planu treba da se pojavi IndexNode i naziv indeksa

idx_insurance_type_sale_date.

Indeksi ubrzavaju čitanje, ali zauzimaju dodatni prostor i usporavaju upis jer se moraju ažurirati nakon svake promene dokumenta.

14. API RUTE

```
GET /api/health
```

Vraća status aplikacije i informaciju da li se koristi ArangoDB ili memorija.

```
GET /api/config
```

Vraća dozvoljene vrste osiguranja.

```
GET /api/clients
```

Vraća korisnike.

```
POST /api/clients
```

Dodaje novog korisnika, zatim sinhronizuje njegovu polisu i graph veze.

GET /api/insurers

Kotva · NoSQL dokumentacija · ArangoDB

Vraća osiguravajuće kuće.

POST /api/insurers

Dodaje novu osiguravajuću kuću.

GET /api/search?q=tekst

Pretražuje korisnike preko analizera, inverted indeksa i search view-a.

GET /api/analytics

Vraća podatke potrebne za grafikone i statističke kartice.

15. VALIDACIJA PODATAKA

Backend proverava:

- da ime i prezime nisu prazni
- da su godine ceo broj između 0 i 120
- da je izabrana dozvoljena vrsta osiguranja
- da datum ima format YYYY-MM-DD
- da osiguravajuća kuća postoji
- da naziv nove kuće ima od 2 do 80 znakova
- da nova kuća nema isti naziv kao postojeća

Neispravan unos vraća HTTP status 400. Duplikat osiguravajuće kuće vraća HTTP status 409.

16. ANALITIKA NA FRONTENDU

Dashboard prikazuje:

- ukupan broj korisnika
- prosečnu starost
- vodeću osiguravajuću kuću
- najjači mesec prodaje putnih polisa
- procenat putnog osiguranja
- prodaju putnih osiguranja po mesecima
- zastupljenost osiguravajućih kuća
- procenat kuće na kružnom grafikonu
- prosečne godine po tipu osiguranja
- strukturu vrsta osiguranja
- dnevnu prodaju putnih polisa tokom jula
- odnos starosti i broja zaključenih polisa

Kružni grafikon je dinamičan. Kada se doda nova kuća, ona se pojavljuje u legendi. Kada dobije prvog korisnika, dobija segment i procenat.

Kotus, NoSQL dokumentacija · ArangoDB

17. PRIMERI AQL UPITA

17.1. Svi korisnici

```
FOR client IN clients

SORT client.surname, client.name

RETURN client
```

17.2. Korisnici auto-osiguranja

```
FOR client IN clients

FILTER client.insuranceType == "Auto"

SORT client.age DESC

RETURN {
```

ime: client.name,

prezime: client.surname,

godine: client.age,

osiguravajucaKuca: client.insurer

```
}
```

17.3. Broj korisnika po kući

```
FOR client IN clients

COLLECT insurer = client.insurer WITH COUNT INTO total

SORT total DESC

RETURN {
```

osiguravajucaKuca: insurer,

brojOsiguranika: total

```
}
```

17.4. Procenat zastupljenosti kuća

```
LET totalClients = LENGTH(clients)

FOR client IN clients
```

```
COLLECT insurer = client.insurer WITH COUNT INTO clientCount
```

```
LET percentage = ROUND(clientCount * 10000 / totalClients) / 100
```

```
SORT clientCount DESC
```

```
RETURN {
```

osiguravajucaKuca: insurer,

brojOsiguranika: clientCount,

procenat: percentage

```
}
```

17.5. Prosečne godine po vrsti osiguranja

```
FOR client IN clients
```

```
COLLECT type = client.insuranceType
```

```
AGGREGATE averageAge = AVERAGE(client.age)
```

```
SORT averageAge DESC
```

```
RETURN {
```

tipOsiguranja: type,

prosecneGodine: ROUND(averageAge * 10) / 10

```
}
```

17.6. Putne polise tokom jula

```
FOR client IN clients
```

```
FILTER client.insuranceType == "Putno"
```

```
FILTER DATE_MONTH(client.saleDate) == 7
```

```
COLLECT day = DATE_DAY(client.saleDate) WITH COUNT INTO policies
```

```
SORT day
```

```
RETURN {
```

dan: day,

brojPolisa: policies

```
}
```

17.7. Najprodavanija vrsta osiguranja

```
FOR client IN clients
```

```
COLLECT type = client.insuranceType WITH COUNT INTO total
```

Kotva · NoSQL dokumentacija · ArangoDB

```
SORT total DESC
```

```
LIMIT 1
```

```
RETURN {
```

tip: type,

brojPolisa: total

```
}
```

17.8. Polise jednog korisnika preko grafa

```
FOR policy IN 1..1 OUTBOUND "clients/korisnik-1" owns
```

```
RETURN policy
```

18. VREMENSKA I MEMORIJSKA KOMPLEKSNOST

Oznake:

- N: broj korisnika
- P: broj polisa
- I: broj osiguravajućih kuća
- V: broj graph čvorova
- E: broj graph veza
- R: broj rezultata upita

Pronalaženje bez indeksa:

```
O(N)
```

Indeksirano pronalaženje:

Približno $O(\log N + R)$

Sortiranje bez odgovarajućeg indeksa:

```
 $O(N \log N)$  vreme i do  $O(N)$  dodatna memorija
```

Agregacija po osiguravajućoj kući:

```
 $O(N + I \log I)$  vreme i  $O(I)$  memorija
```

Prosek starosti:

```
 $O(N)$  vreme
```

Mesečna analitika:

```
 $O(P)$  vreme i  $O(1)$  dodatna memorija, jer postoji 12 meseci
```

Julaska dnevna analitika:

$O(P)$ vreme i $O(1)$ memorija, jer postoje najviše 31 brojac

Kotva · NoSQL dokumentacija · ArangoDB

Graph traversal u najgorem slučaju:

$O(V + E)$

Traversal ograničen na dubinu jedan ili dva obično obilazi samo relevantan deo grafa. Edge indeksi nad `_from` i `_to` ubrzavaju pronalaženje susednih čvorova.

Prostor graph modela:

$O(N + P + E)$

Pošto svaki korisnik trenutno ima jednu polisu i dve veze, ukupna prostorna složenost ostaje linearna $O(N)$, ali uz veći konstantni faktor.

Inverted indeks:

- kreiranje: približno $O(N * L)$, gde je L količina teksta
- pretraga: približno $O(\log T + R)$, gde je T broj termina
- memorija: proporcionalna terminima i referencama ka dokumentima

19. MOGUĆNOSTI OPTIMIZACIJE

1. AQL agregacije

Analitiku računati direktno u bazi, umesto preuzimanja svih dokumenata u Node.js.

2. Jedan prolaz kroz podatke

Više brojača izračunati u jednoj petlji, umesto ponovnog filtriranja za svaku kuću i vrstu.

3. Paginacija

Vraćati npr. 20 korisnika po zahtevu umesto cele kolekcije.

4. Keširanje

Čuvati često korišćene rezultate analitike i osvežavati ih nakon izmene.

5. Preagregirani podaci

Koristiti kolekciju `monthly_statistics` za velike količine podataka.

6. Selektivna graph sinhronizacija

Trenutna demonstraciona implementacija proverava sve graph zapise nakon unosa. Produkcijska verzija bi kreirala samo novu polisu i njene dve veze.

7. Transakcije

Kreiranje korisnika, polise i veza izvršiti kao jednu ACID transakciju.

8. Dodatni indeksi

Dodavati ih samo za često korišćene FILTER i SORT operacije.

10.1.1. NoSQL dokumentacija · ArangoDB

20. SKALIRANJE SISTEMA

Prvi nivo skaliranja:

- optimizacija AQL upita
- indeksi
- paginacija
- keširanje
- preagregirana statistika
- slanje samo potrebnih polja

Horizontalno skaliranje:

- ArangoDB klaster
- sharding kolekcija
- replikacija shardova
- više backend instanci
- load balancer
- zajednički distribuirani keš

Sharding deli različite delove podataka između servera. Replikacija čuva kopije istih podataka radi dostupnosti i oporavka.

Backend treba da bude stateless, odnosno trajno stanje ne treba čuvati u memoriji jedne Node instance.

21. CAP, ACID I BASE

CAP teorema govori o kompromisu distribuiranih sistema između:

- Consistency
- Availability
- Partition tolerance

Trenutni projekat koristi single-server ArangoDB, pa CAP kompromis nije praktično demonstriran. Postao bi važan u klasteru.

ACID:

- Atomicity: sve operacije uspevaju ili se sve poništavaju
- Consistency: podaci ostaju validni
- Isolation: paralelne transakcije se ne ometaju neispravno
- Durability: potvrđeni podaci ostaju trajno sačuvani

BASE se često koristi za opis distribuiranih sistema sa eventual consistency

modelom:

Kotva · NoSQL dokumentacija · ArangoDB

- Basically Available
- Soft state
- Eventual consistency

22. NORMALIZACIJA I DENORMALIZACIJA

Denormalizacija znači namerno ponavljanje podataka radi jednostavnijeg i bržeg čitanja. U client dokumentu postoji naziv insurer iako postoji insurers kolekcija.

Prednosti:

- jednostavniji upiti
- brža analitika

Mane:

- promena naziva može zahtevati izmenu više dokumenata
- mogućnost nekonzistentnosti

```
Graph model client -> policy -> insurer predstavlja strukturisaniji model veza
```

i omogućava više polisa po korisniku.

23. BEZBEDNOST

Za produkcijsku verziju potrebno je:

- promeniti podrazumevanu root lozinku
- ne čuvati lozinke u repozitorijumu
- koristiti environment secrets
- uvesti korisničke uloge
- koristiti TLS
- ograničiti CORS
- uvesti rate limiting
- validirati sve ulazne podatke
- koristiti bind parametre protiv AQL injection napada
- redovno praviti backup

Arangojs aql template koristi bind parametre za dinamičke vrednosti i time smanjuje rizik od AQL injection napada.

24. BACKUP I TRAJNOST

Kotva · NoSQL dokumentacija · ArangoDB

Podaci se čuvaju u Docker volumenu:

arango_data

Volume ostaje nakon komande:

```
docker compose down
```

Volume se briše komandom:

```
docker compose down -v
```

Produkcijski sistem treba da koristi redovne dump/backup procedure i proveru uspešnog vraćanja podataka.

26. PLAN PREZENTACIJE

1. Otvoriti aplikaciju na <http://localhost:3001>.
2. Pokazati statističke kartice i grafikone.
3. Dodati novu osiguravajuću kuću.
4. Dodati korisnika toj kući.
5. Pokazati da se kružni grafikon automatski promenio.
6. Pretražiti korisnika bez dijakritike.
7. Otvoriti ArangoDB na <http://localhost:8529>.
8. Izabrati bazu kotva.
9. Pokazati kolekcije clients, policies i insurers.
10. Pokazati edge kolekcije owns i issued_by.
11. Otvoriti Graphs -> kotva_insurance_graph.
12. Pokazati Analyzer sr_name_search.
13. Pokazati indeks idx_insurance_type_sale_date.
14. Pokrenuti jednostavan AQL upit.
15. Otvoriti Explain i pokazati IndexNode.

27. KRATAK AQL UPIT ZA PREZENTACIJU

```
FOR client IN clients
```

```
  FILTER client.insuranceType == "Auto"
```

```
  SORT client.age DESC
```

```
RETURN {
```

Kotva · NoSQL dokumentacija · ArangoDB

```
ime: client.name,
```

```
prezime: client.surname,
```

```
godine: client.age,
```

```
osiguravajucaKuca: client.insurer
```

```
}
```

Objašnjenje:

Upit prolazi kroz clients kolekciju, zadržava samo korisnike auto-osiguranja,

sortira ih od najstarijeg ka najmlađem i vraća izabrana polja.

28. ZAKLJUČAK

Kotva demonstrira praktičnu upotrebu ArangoDB multi-model baze. Sistem koristi dokumente za korisnike, polise i kuće, edge dokumente za odnose, named graph za vizuelizaciju, analyzer i inverted indeks za pretragu, persistent indeks za filtriranje po vrsti i datumu, kao i AQL za analitiku.

Projekat pokazuje glavne NoSQL koncepte: fleksibilan dokument model, denormalizaciju, indeksiranje, graph traversal, agregaciju, horizontalno skaliranje, sharding, replikaciju i kompromis između brzine čitanja, brzine upisa i zauzeća memorije.